

Q-Crate Design Guide

Architecture, Digital Signal Processing, FPGA, and Firmware

Solomon Tesema

Version 0.1 - August 2026

Contents

1	Preface	3
1.1	Document conventions	3
1.2	Revision history	3
2	Introducing Q-Crate	5
2.1	Motivation	5
2.2	Project goals	5
2.3	First reference application	6
2.4	Non-goals of Version 0.1	6
3	System Architecture	7
3.1	Hardware platform	7
3.2	Architectural partitioning	8
3.3	Control plane and data plane	9
3.4	KV260 block design	10
3.5	Clock architecture	12
3.6	Clock-domain crossing inside qcrate_core	12
3.7	RTL hierarchy	13
3.8	Streaming and DMA architecture	14
3.9	DSP integration	15
3.10	Deterministic sequencer and timebase	16
3.11	R5 real-time control layer	16
3.12	Linux, DMA and memory ownership	17
3.13	Network data path	17
3.14	Interrupt architecture	18
3.15	Reset architecture	19
3.16	Current architecture baseline	19
4	Signal-Processing Foundations	21
4.1	Sampling and aliasing	21
4.2	Real and complex signals	21
4.3	The complex exponential	22
4.4	Baseband, carrier, and local oscillator	22
4.5	Numerically controlled oscillator	22
4.5.1	Phase-to-amplitude implementation choices	23
4.6	Mixing and spectral translation	23
4.7	Digital down-conversion	23
4.8	Digital up-conversion	24
4.9	I/Q constellation	24
4.10	A numerical example	24

5	FPGA Implementation Plan	25
5.1	Proposed datapath decomposition	25
5.2	Initial register-map sketch	25
5.3	AXI DMA data movement	26
5.4	Interrupts	26
5.5	Debug observability	26
6	Firmware and Software Architecture	28
6.1	Processing responsibilities	28
6.2	OpenAMP and RPSMsg	28
6.3	Cache coherency	28
6.4	Python role	29
7	Verification Strategy	30
7.1	Verification levels	30
7.2	NCO verification example	30
7.3	Stream protocol assertions	30
7.4	Hardware acceptance measurements	31
8	Development Roadmap	32
8.1	Proposed milestones	32
8.1.1	Milestone 1 - Infrastructure loop	32
8.1.2	Milestone 2 - NCO	32
8.1.3	Milestone 3 - Digital down-converter	32
8.1.4	Milestone 4 - Heterogeneous control	32
8.1.5	Milestone 5 - External instrumentation	32
8.2	Documentation policy	33
9	Glossary	34
10	Open Questions	35
11	References and Further Reading	36

Chapter 1

Preface

Q-Crate is a learning-driven FPGA instrumentation project with the potential to grow into a reusable high-speed acquisition, signal-processing, and control platform. This guide records both the engineering foundations and the concrete implementation decisions behind the project.

The document has three purposes:

1. to explain the digital-design and signal-processing concepts needed to understand the system;
2. to document the Q-Crate architecture and implementation as they evolve; and
3. to preserve design reasoning, verification methods, and lessons learned.

This is a **living engineering document**. Statements marked *implemented* describe the current design baseline. Statements marked *planned* describe the intended direction and may change after simulation or hardware experiments.

Version 0.1 scope. This first edition establishes the system vocabulary, the current architecture baseline, and the theoretical path from sampled data to complex baseband processing. Later editions will expand the RTL, firmware, verification, and measured-performance chapters alongside development.

1.1 Document conventions

Convention	Meaning
Implemented	Present in the established Q-Crate design baseline
Planned	Intended architecture, not yet claimed as verified hardware
signal_name	RTL signal, register, filename, command, or identifier
$x[n]$	Discrete-time signal indexed by sample number n
f_s	Sampling frequency in samples per second

1.2 Revision history

Version	Date	Summary
0.1	2026-08	Initial architecture and DSP foundations

Chapter 2

Introducing Q-Crate

2.1 Motivation

Modern instrumentation often needs the same combination of capabilities:

- deterministic signal generation and acquisition;
- low-latency digital signal processing;
- software-configurable control and monitoring;
- sustained movement of large sample streams;
- synchronisation between hardware, firmware, and host software; and
- enough observability to debug the complete chain.

Commercial platforms can solve these problems, but they often hide the details that an engineer needs to learn or constrain the system to a fixed workflow. Q-Crate approaches the problem from the opposite direction. It begins as an open engineering platform whose internal data path is deliberately understood, implemented, measured, and documented.

The name **Q-Crate** is currently a project name. The architecture is not tied to a single application. Possible future uses include laboratory data acquisition, waveform generation, quantum-control experiments, LiDAR or radar prototyping, and general FPGA-based instrumentation.

2.2 Project goals

The near-term engineering goals are:

1. establish a clean Zynq UltraScale+ hardware/software architecture;
2. generate and process deterministic streaming data in programmable logic;
3. control the datapath through memory-mapped registers;
4. transfer samples to processor memory through AXI DMA;
5. observe and analyse results from software, including Python tools;
6. develop reusable DSP blocks such as an NCO, mixer, filters, decimators, and interpolators; and
7. verify each block in simulation before integrating it on hardware.

The project is also a structured learning vehicle for SystemVerilog, AXI, DMA, fixed-point DSP, embedded Linux, FreeRTOS, and heterogeneous processing.

2.3 First reference application

The first integrated application is the **Networked Pulsed-IQ Analyzer**. An R5-supervised PL sequence produces deterministic excitation and acquisition events. The DSP path creates a coherent IQ response, DMA moves each triggered shot into explicitly owned memory, and the versioned network data plane sends it to a host for time-domain, magnitude, phase, and spectral analysis.

The first implementation uses a deterministic RTL source and channel model. This is deliberate: it makes every sample and timestamp verifiable before an analogue front end introduces clock, noise, and converter uncertainties. A later ADC replaces the source while preserving the control, timing, DMA, network, recording, and analysis architecture.

The application establishes an important data-integrity rule for future acquisition buffers:

```
FREE -> FILLING -> READY -> USER_OWNED -> FREE
```

Unread measurement data is never overwritten silently. A resource shortage is reported as starvation or a skipped acquisition rather than hidden by stale or torn data.

2.4 Non-goals of Version 0.1

This edition does not claim a finished instrument, a frozen external interface, or measured analogue performance. ADC/DAC selection, RF front-end design, multi-channel synchronisation, and product-level mechanical packaging remain future topics.

Chapter 3

System Architecture

Q-Crate is implemented as a heterogeneous instrumentation system in which processing responsibilities are divided between programmable logic, real-time firmware, embedded Linux, and host software. The first hardware platform is the AMD Kria KV260, built around the Kria K26 system-on-module and its Zynq UltraScale+ MPSoC.

The architecture follows a simple principle: operations whose timing is defined in FPGA clock cycles are performed in programmable logic, bounded real-time supervision is performed by the R5 processor, operating-system services and bulk data movement are handled by the A53 processors running Linux, and computationally heavier analysis and visualisation are performed on the host computer.

This separation is particularly important for instrumentation. Generating a pulse at an exact hardware timestamp, accepting a sample without violating a streaming handshake, allocating a DMA buffer, transmitting a UDP packet, and plotting an FFT are all necessary operations, but they have very different timing requirements. Q-Crate therefore does not attempt to perform them all in one processing environment.

The current reference application, the **Networked Pulsed-IQ Analyzer**, exercises this architecture from deterministic event generation in the programmable logic to IQ analysis on a networked host.

3.1 Hardware platform

The Kria K26 contains a Zynq UltraScale+ MPSoC, which combines several computing resources in one device:

- **programmable logic (PL)** containing FPGA logic, memories and DSP resources;
- **quad-core Arm Cortex-A53 processors**, used by Q-Crate to run embedded Linux;
- **dual Arm Cortex-R5F real-time processors**, of which R5-0 is used by the current Q-Crate real-time control service;
- DDR memory accessible through the processing system;
- PS-to-PL and PL-to-PS AXI interfaces; and
- timers, interrupt controllers, Ethernet and other processing-system peripherals.

The presence of both processors and programmable logic allows Q-Crate to be partitioned according to timing requirements rather than according to programming language.

The PL is responsible for the deterministic signal and timing path. A hardware action in this domain can be related directly to the 200 MHz system clock, whose period is

$$T_{\text{clk}} = \frac{1}{200 \text{ MHz}} = 5 \text{ ns.}$$

Consequently, a timing event separated by 100 clock ticks represents an interval of exactly 500 ns in the digital timing model.

Linux on the A53 processors operates on a completely different timescale. It provides memory management, device drivers, filesystems, Ethernet networking and user applications, but normal Linux task scheduling does not guarantee execution on a particular 5 ns clock boundary. Q-Crate therefore uses Linux for supervisory and transport functions rather than for waveform-level timing.

The R5 processor occupies an intermediate position. It executes software, but its FreeRTOS environment is substantially more deterministic than general-purpose Linux. In the present architecture R5-0 supervises the pulse sequencer and provides the software ownership boundary for sequence upload and lifecycle operations.

3.2 Architectural partitioning

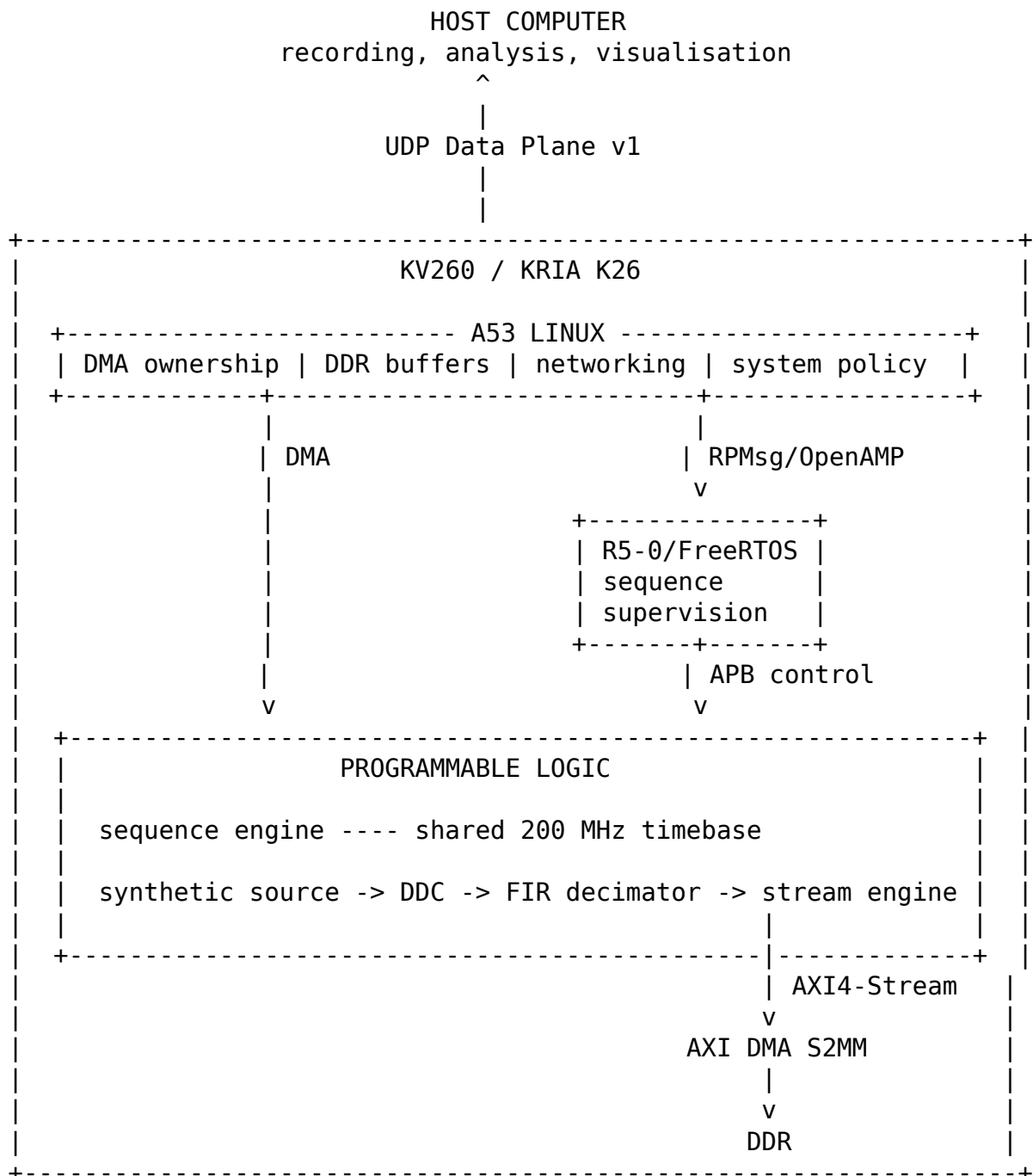
The implemented Q-Crate system can be viewed as four cooperating execution domains.

Domain	Main responsibility	Timing character
Programmable logic	Sequencing, timebase, DSP and AXI4-Stream generation	Clock deterministic
R5-0 / FreeRTOS	Real-time sequence supervision and validation	Bounded real-time software
A53 / Linux	DMA, memory-buffer ownership, networking and system services	General-purpose software
Host computer	Configuration, recording, visualisation and numerical analysis	Non-real-time

These domains form a hierarchy rather than four independent computers.

At the lowest level, programmable logic executes events and produces samples. R5 controls the deterministic sequence without generating the individual pulse timings in software. Linux manages the much larger memory and networking environment around each acquisition. Finally, the host receives measurement records and interprets them as time-domain or frequency-domain data.

A simplified end-to-end view is:



The diagram also shows an important distinction between **supervision** and **execution**. R5 supervises a sequence, but the PL sequence engine executes its event timings. Linux manages an acquisition, but the PL and DMA move the actual sample stream without depending on Linux to service every individual sample.

3.3 Control plane and data plane

Q-Crate separates low-bandwidth configuration traffic from high-bandwidth measurement traffic.

These two traffic classes are referred to as the **control plane** and the **data plane**.

The control plane carries commands and state. Examples include loading a sequence, setting a frame length, starting an acquisition, reading a status register and acknowledging an interrupt.

The internal data plane carries sample streams. Once an acquisition begins, potentially millions of sample values must move through the system without requiring a processor transaction for every sample.

Property	Control plane	Internal data plane
Information	Commands, configuration and status	Measurement samples
Typical traffic	Occasional register accesses	Sustained stream
PL interface	APB	AXI4-Stream
Memory transfer	Not normally required	AXI DMA
Primary concern	Correct command semantics	Throughput and back-pressure

This separation is fundamental to the architecture. A processor may require many clock cycles or even microseconds to issue a software command, whereas an FPGA datapath can accept or produce a value on every 200 MHz clock cycle.

Attempting to control individual samples through software-visible registers would therefore couple measurement throughput to processor latency. Q-Crate instead configures a hardware operation through the control plane and allows the resulting sample sequence to travel independently through the streaming data path.

At the network boundary the term *data plane* is also used for **Q-Crate Data Plane v1**, the versioned UDP wire protocol used between the KV260 and the host. This network protocol is distinct from AXI4-Stream, although both participate in transporting the same measurement data at different stages of the system.

3.4 KV260 block design

The Vivado block design provides the infrastructure connecting the Zynq processing system, Q-Crate RTL and DDR memory.

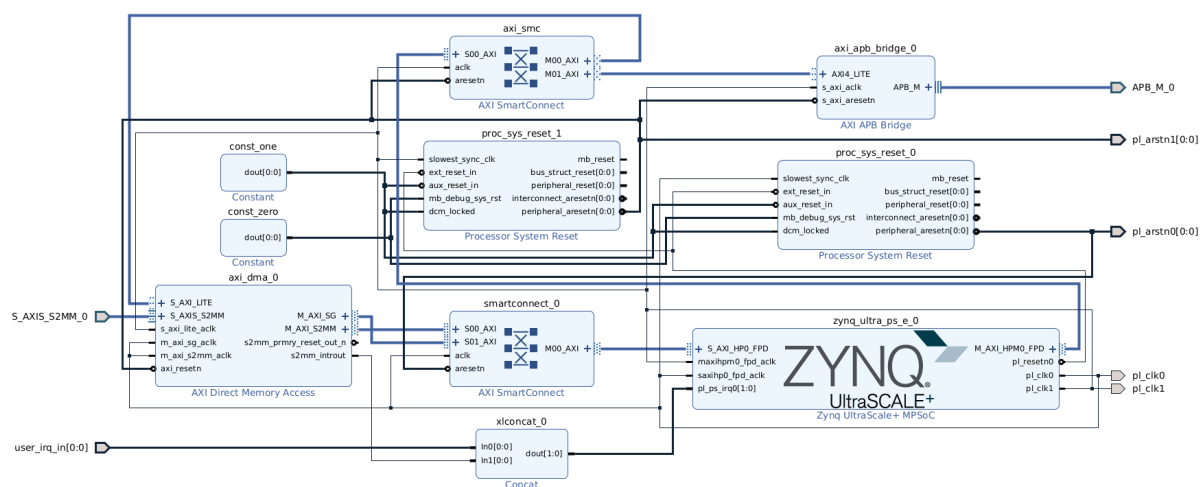
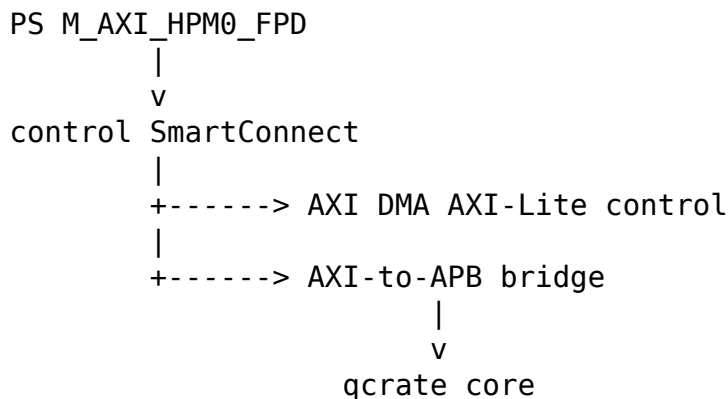


Figure 3.1: Current Q-Crate KV260 Vivado block design

Figure: Current reproducible KV260 Vivado block design. The corresponding exported `design_1.tcl` file is the reproducible source representation of the diagram.

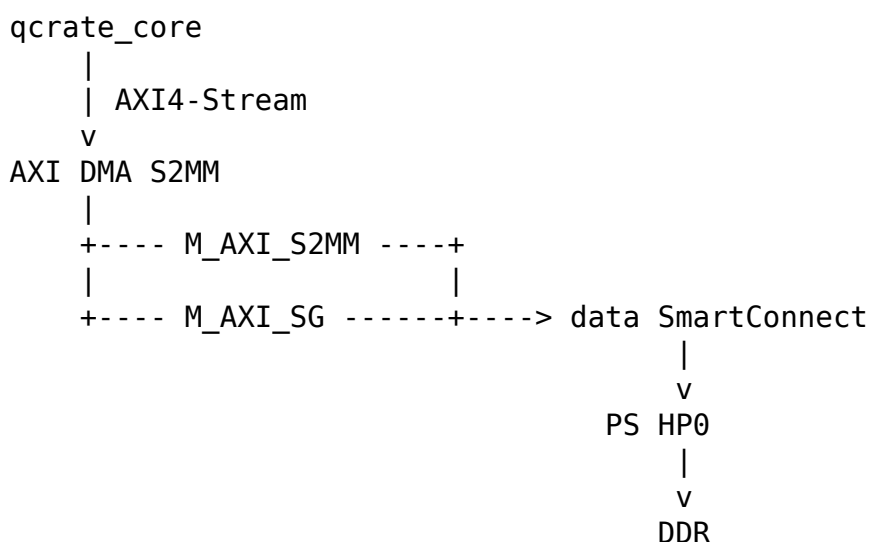
The block design contains two major paths: a processor-controlled register path and a DMA payload path.

The control path is:



The processor accesses the custom Q-Crate register space through an AXI-to-APB bridge. The AXI side belongs to the Zynq processing-system infrastructure; the APB side is implemented as part of the Q-Crate RTL interface.

The DMA payload path is:



S2MM means *stream to memory-mapped*. Samples arrive at the DMA as an AXI4-Stream and are written to addresses in DDR memory.

The DMA also uses an independent scatter-gather master interface, `M_AXI_SG`, to read DMA descriptors. This allows a sequence of memory buffers to be prepared before an acquisition begins instead of requiring software to reprogram the DMA after every frame.

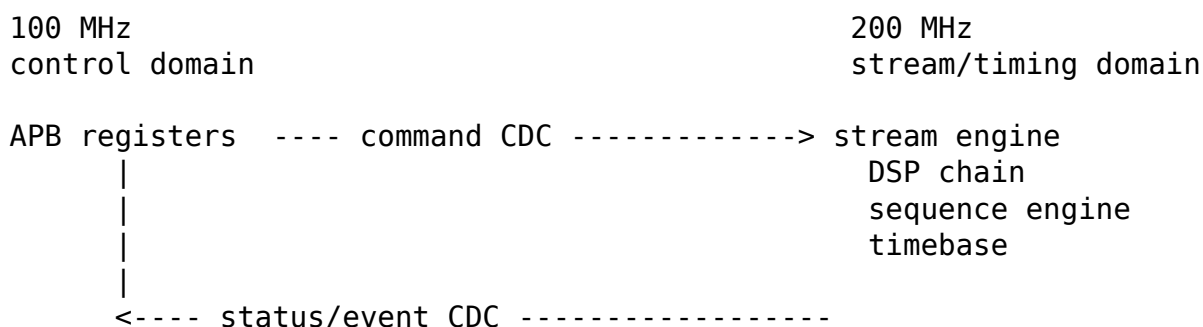
The payload and descriptor interfaces share the PS high-performance memory path through the data SmartConnect.

3.5 Clock architecture

Two primary PL clock domains are used.

The **control domain** operates at 100 MHz and is driven by `pl_clk1`. APB transactions and most software-visible register logic operate in this domain.

The **stream and timing domain** operates at 200 MHz and is driven by `pl_clk0`. The sample pipeline, AXI4-Stream source, DSP chain, sequence engine and 64-bit hardware timebase operate in this domain.



The two domains exist because their tasks have different requirements. Control transactions do not benefit from operating at the full signal-processing clock rate, while the datapath and timing engine require the higher-frequency domain.

An earlier form of the Q-Crate block design included an external AXI Clock Converter in the control path. The current architecture does not require it. The processor-side control SmartConnect, DMA AXI-Lite interface and AXI-to-APB bridge all operate in the 100 MHz control domain.

The AXI DMA is the boundary between the 100 MHz control interface and the 200 MHz streaming interface. It is configured for asynchronous operation so that its control and payload interfaces can use the appropriate clocks without an additional external converter in the APB branch.

The custom RTL nevertheless contains genuine clock-domain crossings because commands and status must travel between the 100 MHz APB domain and the 200 MHz execution domain.

3.6 Clock-domain crossing inside `qcrate_core`

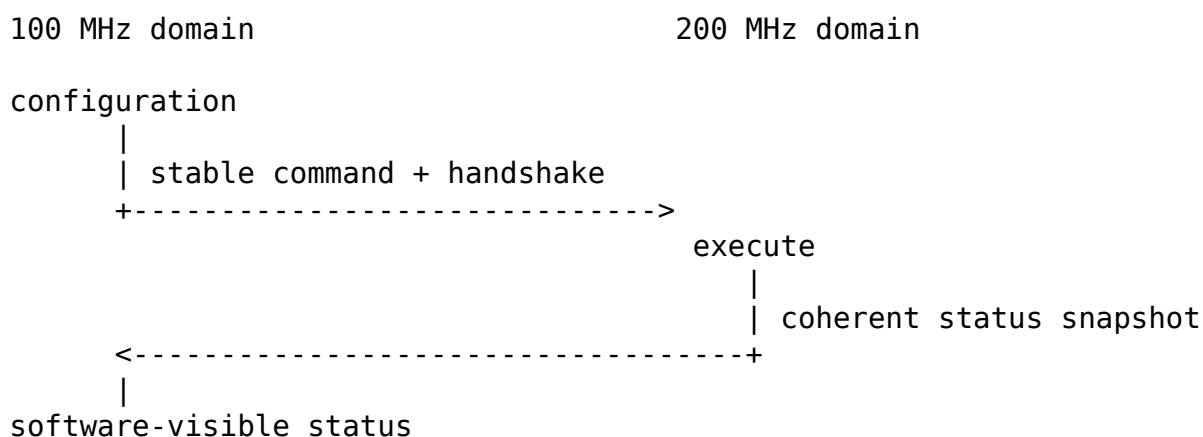
A signal sampled by logic driven from an unrelated clock can violate setup or hold requirements and enter a metastable state. Clock-domain crossing, usually abbreviated **CDC**, is therefore an architectural concern rather than merely a wiring detail.

Q-Crate uses different CDC structures according to the type of information being transferred.

Single-bit asynchronous state uses synchronising flip-flops. Command transactions carrying several related fields use a handshake in which the source retains the complete value while an event indication crosses the boundary. Status information moving in the opposite direction is transferred as a coherent snapshot rather than as independently synchronised bits.

The sequencer also uses memory that is accessible from both clock domains. Software-visible accesses occur from the control side, while the sequence engine reads events from the 200 MHz execution side.

The result is a clear ownership model:



This prevents a multi-bit value such as a frame length, event count or timestamp from being observed as a mixture of old and new bits.

3.7 RTL hierarchy

The handwritten programmable-logic design is implemented primarily in SystemVerilog.

The top-level integration module is:

`kv260/hw/rtl/qcrate_top.sv`

`qcrate_top` connects the Vivado-generated block design to the custom Q-Crate RTL. Its principal interfaces are the APB control bus, AXI4-Stream output, the two PL clocks and resets, and the interrupt connection back toward the processing system.

Below this integration layer is:

`kv260/hw/rtl/qcrate_core.sv`

`qcrate_core` contains the main instrument logic. Its major subsystems can be represented as:

```

qcrate_core
|
+-- APB fabric
|   |
|   +-- system registers
|   +-- stream registers
|   +-- sequence registers
|
+-- control/stream CDC
|
+-- stream subsystem
|   |
|   +-- DSP chain

```

```

|   +-- stream engine
|
+-- deterministic sequence subsystem
|   |
|   +-- sequence event RAM
|   +-- sequence engine
|   +-- 64-bit timebase
|
+-- status/event CDC
|
+-- interrupt controller

```

The APB fabric divides the Q-Crate register space into 4 KiB pages. Page zero contains system identification and general registers, page one contains stream-control registers, and page two contains sequencer registers and access to sequence-event memory.

The SystemVerilog hierarchy separates the custom instrument logic from the Vivado block design. The generated design therefore provides processing-system and AXI infrastructure, while modules such as the sequencer, stream engine, CDC logic and register banks remain ordinary RTL that can be simulated independently.

Earlier Q-Crate sources used the filename `qcrate_top.v`. The active implementation is now `qcrate_top.sv`, and both the top-level integration and the custom core are SystemVerilog modules.

3.8 Streaming and DMA architecture

The stream engine exposes a 32-bit AXI4-Stream master interface to the DMA.

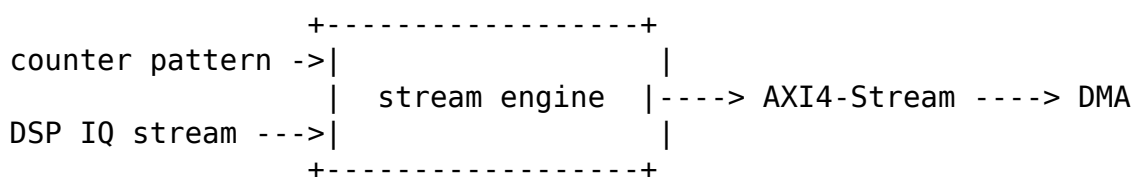
For every transfer, the interface includes:

- TDATA, containing the sample word;
- TKEEP, indicating valid bytes;
- TVALID, asserted when the source presents valid data;
- TREADY, asserted when the DMA can accept data; and
- TLAST, marking the final word of a frame.

A transfer occurs only on a clock edge for which both TVALID and TREADY are asserted.

This handshake allows downstream back-pressure to propagate to the source. If the DMA temporarily cannot accept another word, the Q-Crate stream engine holds the pending transfer rather than discarding the sample.

The stream engine currently supports two sources. A deterministic counter-pattern mode was retained because it provides a simple method of verifying framing and DMA integrity. The second mode selects the integrated DSP output.



The pattern source is useful for determining whether corruption originates in the data transport system rather than in DSP arithmetic. Once the DMA path has been verified independently, the same framing mechanism can carry DSP output.

3.9 DSP integration

The Q-Crate DSP chain is part of the implemented architecture.

The present signal source is synthetic rather than an external ADC. It produces deterministic digital input samples so that the complete acquisition chain can be compared against a bit-accurate software model.

The current processing path is:

synthetic sampled source

|

v

digital down-conversion

|

v

complex I/Q

|

v

FIR decimation by 16

|

v

16-bit I + 16-bit Q

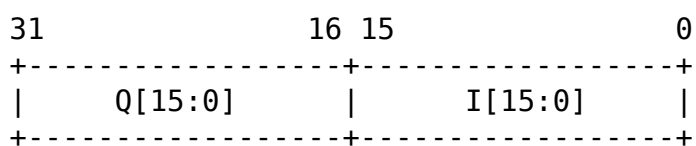
|

v

32-bit AXI4-Stream word

The source operates in the 200 MHz stream domain. Digital down-conversion translates the selected spectral component to complex baseband, after which the FIR stage performs low-pass filtering and decimation.

The packed output word contains a signed 16-bit I value and a signed 16-bit Q value:



The implemented DSP chain has been captured through the KV260 DMA path and compared with its bit-accurate Python reference model. The hardware and model produced matching captured samples for the accepted DSP test configuration.

The synthetic source is therefore not an architectural placeholder for an unfinished DSP subsystem. It is the current digital stimulus at the front of an implemented DSP and acquisition path. A physical ADC can later replace this source while leaving the downstream DDC, filtering, framing, DMA and software interfaces structurally intact.

3.10 Deterministic sequencer and timebase

Q-Crate contains a hardware event sequencer in the 200 MHz timing domain.

A sequence consists of timestamped events stored in dual-clock event memory. Each event describes a required output state at a particular offset from the beginning of the sequence.

Before execution, the event sequence is loaded and validated. During execution, the PL compares the current elapsed time with the event timestamps and updates the sequence outputs at the specified hardware ticks.

A 64-bit counter provides the common hardware timebase. At 200 MHz, each increment represents 5 ns.

The important architectural separation is:

R5 firmware:

```
load
validate
arm
start
abort
inspect status
```

PL sequencer:

```
count hardware ticks
fetch events
compare timestamps
change outputs
```

The R5 does not wake up every 5 ns to generate a pulse. It establishes and supervises the sequence; the FPGA executes the timing.

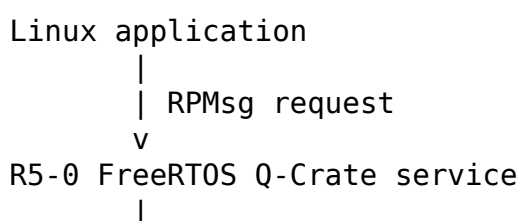
The present sequencer supports two logical pulse outputs. In the current KV260 top-level design these signals remain internal to the programmable-logic integration rather than being committed to a final external instrument connector. Their timing engine and control path are nevertheless implemented and verified.

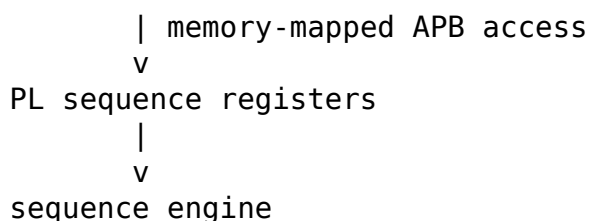
3.11 R5 real-time control layer

R5-0 runs a FreeRTOS-based Q-Crate control service.

Communication between Linux and the R5 uses **OpenAMP/RPMsg**. OpenAMP provides the heterogeneous-processing communication framework, while RPMsg provides message-based communication between the Linux and remote-processor environments.

The operational relationship is:





The R5 service directly accesses the Q-Crate register space and owns the sequence upload and lifecycle operations. It verifies sequence information, writes event memory, issues arm/start/abort commands and reads sequencer status.

This design keeps real-time sequence supervision outside the general-purpose Linux scheduler while retaining Linux as the system-management environment.

OpenAMP was initially developed as a vertical integration experiment within Q-Crate, but it is now part of the implemented control architecture used by the reference application.

3.12 Linux, DMA and memory ownership

Linux on the A53 processors owns the bulk acquisition infrastructure.

The AXI DMA writes completed PL frames into DDR memory. Scatter-gather operation allows Linux to construct a chain of descriptors describing multiple destination buffers before the stream begins.

This arrangement separates two kinds of activity:

PL / DMA	Linux
produce samples	-----> receive into prepared memory
execute framing	own DMA descriptors
assert TLAST	manage completed buffers
	package measurement records
	send data over Ethernet

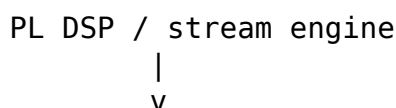
Linux therefore does not copy each FPGA sample individually as it arrives. The DMA engine performs the high-rate transfer into memory, after which software operates on completed memory regions.

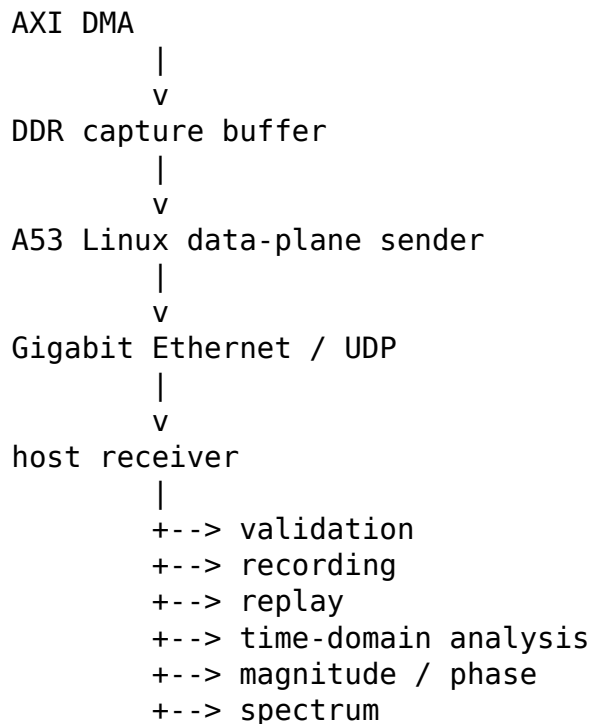
The finite-shot implementation uses coherent DMA buffers and scatter-gather descriptor chains. This provides the basis for repeated acquisition in which buffer ownership must remain explicit while one part of the system fills memory and another part transmits previously completed measurements.

3.13 Network data path

After a DMA capture is complete, the measurement is transferred from the KV260 to the host through Ethernet.

The implemented path is:





Q-Crate Data Plane v1 defines the binary UDP representation used for this transfer. The protocol was frozen after end-to-end KV260-to-host acceptance of the finite acquisition path.

The network protocol does not replace the internal AXI4-Stream interface. Instead, the complete acquisition crosses several interfaces according to where the data is located:

DSP	AXI4-Stream
PL -> DDR	AXI DMA
DDR -> Linux	memory / DMA driver
Linux -> host	UDP Data Plane v1

Each interface solves a different part of the transport problem.

3.14 Interrupt architecture

The programmable logic provides an interrupt toward the processing system for events that require software attention.

Within `qcrate_core`, event pulses generated in the 200 MHz execution domain are transferred into the control domain and converted into software-visible interrupt state. Interrupt events are stored as sticky status rather than requiring software to observe a single-cycle FPGA pulse directly.

This distinction is important because a one-cycle pulse at 200 MHz lasts only 5 ns, far shorter than software polling intervals. The hardware therefore records the occurrence of the event until software acknowledges it.

The interrupt mechanism complements status registers: status describes the present state of a subsystem, whereas an interrupt can record that a significant event occurred.

3.15 Reset architecture

The two principal clock domains have corresponding active-low reset signals:

```
pl_clk0    200 MHz stream/timing clock
pl_arstn0  stream/timing reset
```

```
pl_clk1    100 MHz control clock
pl_arstn1  control reset
```

The generated block design contains the processor-system reset infrastructure used to derive resets associated with the PL clocks.

Reset behaviour is significant in a heterogeneous design because the processors, AXI infrastructure, DMA and custom logic cannot be treated as a single flip-flop network. Each subsystem must enter a defined state, and reset release must respect the clock domain in which the receiving logic operates.

The AXI DMA spans the control and payload domains internally. Its external reset is associated with the slower control side, while the DMA IP performs the required internal handling for its asynchronous clock configuration.

Within `qcrate_core`, the control and stream reset inputs independently initialise the state belonging to their respective clock domains.

3.16 Current architecture baseline

The principal implemented elements of the Q-Crate KV260 architecture are summarised below.

Element	Current implementation
Hardware platform	AMD Kria KV260 / K26 Zynq UltraScale+ MPSoC
PL control clock	100 MHz
PL stream/timing clock	200 MHz
Custom control bus	32-bit APB
Processor-to-control path	AXI SmartConnect and AXI-to-APB bridge
Stream transport	32-bit AXI4-Stream
Memory transfer	AXI DMA S2MM with scatter-gather
DDR access	PS HP0 path through data SmartConnect
Top-level RTL	<code>qcrate_top.sv</code>
Main custom RTL	<code>qcrate_core.sv</code>
Register structure	System, stream and sequence APB pages
CDC	Explicit command, event and status crossings
Timing	64-bit, 200 MHz hardware timebase
Sequencing	Deterministic two-channel event sequencer
DSP	Synthetic source, DDC, complex I/Q and decimate-by-16 FIR
Real-time processor	R5-0 running FreeRTOS
A53/R5 communication	OpenAMP / RPMsg
A53 software	Embedded Linux / PetaLinux
DMA software	Linux DMAEngine-based capture
Network transport	Gigabit Ethernet, UDP Data Plane v1
Host software	Receiver, capture recording, replay and DSP analysis

This architecture forms the present digital foundation of Q-Crate. The synthetic signal source and internal pulse outputs allow the timing, DSP, control, DMA and networking layers to be developed and measured without yet depending on an analogue front end.

A later physical acquisition interface changes the boundary at which real-world signals enter the system, but it does not fundamentally alter the division between deterministic programmable logic, real-time supervision, Linux data management and host-level analysis.

Chapter 4

Signal-Processing Foundations

4.1 Sampling and aliasing

A continuous signal sampled at frequency f_s produces the sequence

$$x[n] = x_a(nT_s), \quad T_s = \frac{1}{f_s}.$$

Frequencies separated by integer multiples of f_s produce identical sampled sequences. A sinusoid at f therefore aliases with signals at $f + kf_s$. To uniquely represent a baseband signal limited to B hertz, the classical Nyquist condition requires $f_s > 2B$, together with a practical analogue anti-alias filter.

The discrete-time frequency axis repeats every f_s . A convenient principal interval is $[-f_s/2, f_s/2)$.

4.2 Real and complex signals

A real signal has a conjugate-symmetric Fourier spectrum:

$$X(-f) = X^*(f).$$

Its negative-frequency content is therefore not independent. A complex signal

$$z[n] = I[n] + jQ[n]$$

can represent positive and negative frequencies independently. This is one of the main reasons complex signals are so useful in communication and instrumentation DSP.

I means **in-phase** and Q means **quadrature**. Q is not a second unrelated measurement; it is the component along an axis 90 degrees from I in the complex plane.

4.3 The complex exponential

Euler's identity gives

$$e^{j\theta} = \cos \theta + j \sin \theta.$$

A sampled complex oscillator is therefore

$$z[n] = Ae^{j(2\pi f_0 n/f_s + \phi_0)}.$$

Unlike a real cosine, this complex exponential occupies one spectral direction. Multiplication by it translates a spectrum without creating the same mirrored pair produced by a real mixer.

4.4 Baseband, carrier, and local oscillator

Baseband is the frequency region around zero that contains the information of interest. A radio-frequency or intermediate-frequency carrier moves that information away from zero for transmission, acquisition, or analogue processing.

A **local oscillator (LO)** provides the reference used by a mixer to translate frequencies. In a digital system the LO is often generated by an NCO. Mixing a signal with $e^{-j2\pi f_{LO}n/f_s}$ shifts spectral content at f_{LO} toward zero:

$$y[n] = x[n]e^{-j2\pi f_{LO}n/f_s}.$$

If the desired channel was centred at f_{LO} , it is now at complex baseband.

4.5 Numerically controlled oscillator

An NCO commonly contains:

1. a phase accumulator;
2. a phase-to-amplitude converter; and
3. optional amplitude scaling.

For a P -bit phase accumulator,

$$\Phi[n+1] = (\Phi[n] + \text{FTW}) \bmod 2^P,$$

where FTW is the frequency tuning word. The generated frequency is

$$f_{out} = \frac{\text{FTW}}{2^P} f_{clk}.$$

The frequency resolution is

$$\Delta f = \frac{f_{clk}}{2^P}.$$

For example, with a 32-bit accumulator clocked at 200 MHz, the theoretical frequency step is approximately 0.0466 Hz. This does not mean the output has perfect spectral purity: phase truncation, amplitude quantisation, and lookup table structure create spurs and noise.

4.5.1 Phase-to-amplitude implementation choices

Method	Advantages	Costs
LUT/ROM	Simple, predictable latency	Memory and quantisation spurs
CORDIC	Flexible magnitude/phase operations	Iterations, latency, logic
Vendor DDS IP	Fast integration and characterised options	Less transparent and portable

An educational Q-Crate implementation should begin with a small, explicit NCO whose phase and amplitude widths can be inspected. A vendor IP version can later provide a performance and resource comparison.

4.6 Mixing and spectral translation

Multiplication in time corresponds to convolution in frequency. Multiplying by a complex exponential shifts the spectrum:

$$x[n]e^{j2\pi f_0 n/f_s} \iff X(f - f_0).$$

For real input $x[n]$ and an NCO producing cosine and sine, a down-converter can form

$$I[n] = x[n] \cos \theta[n], \quad Q[n] = -x[n] \sin \theta[n].$$

The sign convention must be consistent across RTL, software, plots, and tests. Changing the sign of Q reverses the apparent spectral direction.

4.7 Digital down-conversion

A DDC converts a selected band to a lower sample-rate complex baseband stream. Its conceptual stages are:

1. complex mixing with an NCO;
2. low-pass filtering to retain the wanted channel;
3. decimation to remove unnecessary samples; and
4. gain, formatting, and optional correction.

Filtering must precede or be integrated with decimation. Simply keeping every R th sample allows out-of-band energy to alias into the new Nyquist interval.

For large decimation ratios, efficient implementations may combine CIC and FIR filters. The CIC performs multiplier-free rate reduction but introduces passband droop; a compensation FIR can correct it.

4.8 Digital up-conversion

A DUC performs the reverse path:

1. interpolate the baseband signal;
2. filter the inserted spectral images;
3. mix the complex signal to the desired carrier; and
4. scale and format the result for a DAC.

Interpolation by L inserts $L - 1$ zeros between original samples before the interpolation filter reconstructs intermediate values. Repeating samples is a different operation and produces a zero-order-hold response.

4.9 I/Q constellation

A constellation plot places I on the horizontal axis and Q on the vertical axis. For communication waveforms, clusters correspond to symbols. For Q-Crate, the same plot is useful beyond communications: it can reveal phase rotation, amplitude variation, quadrature imbalance, DC offset, noise, and saturation.

Magnitude and phase follow from

$$|z| = \sqrt{I^2 + Q^2}, \quad \angle z = \text{atan2}(Q, I).$$

The atan2 form is essential because it preserves the correct quadrant.

4.10 A numerical example

Assume a real 25 MHz sinusoid sampled at 100 MSPS. A DDC NCO is programmed to 25 MHz and the mixer uses the negative complex exponential. The desired tone is translated to 0 Hz. A low-pass filter removes the high-frequency mixing product, and the result is a nearly constant complex value whose magnitude and angle encode the input amplitude and phase.

If the input instead lies at 25.1 MHz, the complex baseband output rotates at 100 kHz. This rotating phasor is often a more useful representation than the original high-frequency waveform.

Chapter 5

FPGA Implementation Plan

5.1 Proposed datapath decomposition

The first reusable DSP chain should be divided into independently verifiable blocks:

Module	Responsibility	Initial verification focus
qcrate_nco	Phase accumulation and sine/cosine generation	Frequency and phase continuity
qcrate_mixer	Real-to-complex or complex mixing	Sign, scaling, pipeline alignment
qcrate_fir	Channel filtering	Impulse and frequency response
qcrate_decimator	Rate reduction	Output cadence and alias rejection
qcrate_axis_pack	Format samples for DMA	TVALID/TREADY/TLAST correctness
qcrate_regs	Configuration and status	Address and field semantics

Each block should expose explicit sample validity. Configuration changes such as a new FTW should have documented timing: immediate, sample-boundary, or frame-boundary application.

5.2 Initial register-map sketch

This table is a proposal, not a frozen interface.

Offset	Name	Access	Purpose
0x00	CONTROL	R/W	Core enable, soft reset, stream enable
0x04	STATUS	R/O	Running, overflow, error flags
0x08	NCO_FTW	R/W	NCO frequency tuning word
0x0C	NCO_PHASE	R/W	Initial or commanded phase offset
0x10	DECIMATION	R/W	Requested decimation factor
0x14	FRAME_LENGTH	R/W	Samples per DMA frame
0x18	SAMPLE_COUNT	R/O	Accepted output samples

Offset	Name	Access	Purpose
0x1C	ERROR_COUNT	R/O	Overflow or protocol error count

Reserved bits should read as zero and ignore writes. Pulse-like commands should not rely on software returning a bit to zero unless that behaviour is deliberate.

5.3 AXI DMA data movement

DMA prevents the CPU from copying every sample. The PL produces an AXI4-Stream; the S2MM channel writes it into DDR. Software configures buffer addresses and lengths, starts the transfer, and responds to completion or error events.

The practical design must address:

- physically contiguous or DMA-capable buffers;
- cache coherency and cache maintenance;
- frame length and TLAST semantics;
- buffer ownership between DMA and software;
- interrupt versus polling operation;
- continuous operation using multiple or cyclic buffers; and
- recovery from halted or overflow conditions.

The peak payload rate of a stream is approximately

$$R = f_{axis} \times \frac{W_{data}}{8} \times \eta,$$

where η is the fraction of cycles on which a transfer occurs. A 64-bit stream at 200 MHz has a theoretical interface payload of 1.6 GB/s at $\eta = 1$, but actual memory throughput will be lower and must be measured.

5.4 Interrupts

The current block design provides two PL-to-PS interrupt lines. Candidate uses include DMA completion/error and custom-core status. Interrupt sources should remain asserted until software can identify and acknowledge them; a one-cycle pulse may be missed by the processor-side interrupt controller.

5.5 Debug observability

Every major integration stage should provide observability without requiring a re-design. Useful probes and counters include:

- accepted input and output sample counts;
- stalled-cycle count;
- FIFO occupancy and overflow/underflow flags;
- current configuration snapshot;
- frame and interrupt counters; and
- sticky first-error status.

ILA is valuable for short, precisely triggered events, but architectural status counters are better for long-running behaviour and field diagnostics.

Chapter 6

Firmware and Software Architecture

6.1 Processing responsibilities

The eventual software architecture may use both processor classes.

Compute domain	Suitable responsibilities
Programmable logic	Per-sample DSP, framing, deterministic timing
R5 / FreeRTOS	Deterministic control loops, peripheral management
A53 / Linux	Networking, files, Python, UI, orchestration

This is a guide rather than a requirement that every feature use all three domains. Complexity should be added only when it solves a measured need.

6.2 OpenAMP and RPMsg

OpenAMP is a framework for communication and lifecycle management in asymmetric multiprocessing systems. On Zynq UltraScale+, it can support a Linux application on the A53 communicating with firmware running on an R5.

RPMsg provides logical message channels, while shared memory carries the data behind those messages. OpenAMP is appropriate for commands, status, and moderate message traffic between operating environments. It should not be confused with AXI DMA: DMA moves bulk data between hardware and memory, whereas OpenAMP coordinates software running on different processors.

6.3 Cache coherency

A completed DMA interrupt does not automatically guarantee that software reads fresh bytes from a cached mapping. Depending on the memory path and software configuration, buffers may require cache invalidation before CPU reads and cache flushing before a DMA device reads CPU-produced data.

Buffer ownership must be explicit:

1. software prepares a buffer;
2. ownership passes to DMA;
3. DMA completes and returns ownership;
4. software performs required cache maintenance; and
5. software consumes the data.

6.4 Python role

Python is intended for control, experiment automation, visualisation, and reference-model comparison. It is not part of the deterministic per-sample path. Early Python tools should:

- configure registers;
- capture a finite sample frame;
- decode the packed sample format;
- plot time-domain, spectrum, and I/Q views; and
- compare hardware output with a NumPy reference model.

Chapter 7

Verification Strategy

7.1 Verification levels

Q-Crate verification should progress through four levels.

1. **Mathematical reference:** floating- and fixed-point Python models.
2. **RTL unit tests:** self-checking SystemVerilog tests for individual blocks.
3. **RTL integration:** stream back-pressure, control updates, resets, and module interaction.
4. **Hardware validation:** ILA captures, DMA frames, spectral measurements, and long-duration stress tests.

Passing one level does not replace the next. A correct floating-point model does not prove fixed-point widths, and a passing RTL simulation does not prove timing closure or analogue signal integrity.

7.2 NCO verification example

An NCO testbench should verify more than the appearance of a sine wave.

- FTW-to-frequency relationship;
- phase increment on every accepted sample;
- wrap-around behaviour;
- phase-offset loading;
- output range and signedness;
- stable output during downstream back-pressure, if streamed;
- deterministic reset behaviour; and
- expected error against a high-precision reference.

Spectral tests should use coherent sampling where possible. If the captured record contains an integer number of cycles, leakage is reduced and spurious components are easier to distinguish from windowing artefacts.

7.3 Stream protocol assertions

Useful assertions include:

- payload remains stable while TVALID && !TREADY;
- TLAST appears at the intended frame position;

- accepted-sample count matches the reference model;
- no FIFO overflow or underflow occurs; and
- reset returns the interface to a defined idle state.

7.4 Hardware acceptance measurements

Each integrated milestone should record:

Category	Example measurement
Functional	Correct sample values and ordering
Throughput	Sustained MB/s and stall percentage
Timing	Worst negative slack and clock constraints
Resources	LUT, FF, BRAM, URAM, DSP utilisation
Signal quality	SNR, SFDR, amplitude and phase error
Reliability	Duration and data volume without error

Chapter 8

Development Roadmap

8.1 Proposed milestones

8.1.1 Milestone 1 - Infrastructure loop

- Finalise APB register access.
- Generate a deterministic counter or pattern in `qcrate_core`.
- Stream frames through AXI DMA into DDR.
- Verify the captured data from software.

8.1.2 Milestone 2 - NCO

- Implement a parameterised SystemVerilog NCO.
- Build a self-checking testbench and Python reference.
- Stream sine/cosine samples through DMA.
- Measure frequency accuracy and spectral purity.

8.1.3 Milestone 3 - Digital down-converter

- Add mixer and low-pass filtering.
- Add programmable decimation.
- Capture I/Q baseband samples.
- Validate spectrum, magnitude, and phase in Python.

8.1.4 Milestone 4 - Heterogeneous control

- Define whether an R5 control task is justified.
- If justified, introduce FreeRTOS and OpenAMP/RPMsg.
- Keep bulk samples on the DMA path.

8.1.5 Milestone 5 - External instrumentation

- Integrate selected ADC/DAC hardware.
- Define clock, trigger, and synchronisation architecture.
- Measure real analogue and end-to-end performance.

8.2 Documentation policy

The guide should evolve in the same commits as the design:

- an interface change updates its table and diagram;
- a new RTL block adds its numerical format and verification method;
- a measured result records the setup and tool versions;
- a rejected approach is retained briefly in a design-decision note; and
- planned features remain clearly separated from verified implementation.

Chapter 9

Glossary

Term	Meaning
ADC	Analogue-to-digital converter
APB	Advanced Peripheral Bus
AXI	Advanced eXtensible Interface
CDC	Clock-domain crossing
CIC	Cascaded integrator-comb filter
DAC	Digital-to-analogue converter
DDC	Digital down-converter
DMA	Direct memory access
DSP	Digital signal processing
DUC	Digital up-converter
FIR	Finite impulse response filter
FTW	Frequency tuning word
I/Q	In-phase and quadrature components
IF	Intermediate frequency
LO	Local oscillator
NCO	Numerically controlled oscillator
PL	Programmable logic
PS	Processing system
RF	Radio frequency
RPMsg	Remote processor messaging
S2MM	Stream to memory-mapped DMA direction
SFDR	Spurious-free dynamic range
SNR	Signal-to-noise ratio

Chapter 10

Open Questions

The following decisions are intentionally left open for future revisions:

1. What is the first external ADC/DAC configuration?
2. What sample word format and frame metadata should become the stable host API?
3. Which configuration changes require atomic or frame-boundary application?
4. What continuous DMA buffering strategy best fits the chosen Linux stack?
5. Which functions genuinely benefit from the R5 and OpenAMP?
6. What clock and trigger interfaces are required for multi-channel operation?
7. Which performance targets define a successful Q-Crate instrument?

Chapter 11

References and Further Reading

The next revision should attach exact document versions to each reference.

- AMD, *Zynq UltraScale+ MPSoC Technical Reference Manual*.
- AMD, *AXI DMA Product Guide*.
- AMD, *AXI Reference Guide*.
- ARM, *AMBA APB Protocol Specification*.
- OpenAMP Project documentation.
- A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*.
- R. G. Lyons, *Understanding Digital Signal Processing*.
- F. J. Harris, *Multirate Signal Processing for Communication Systems*.